Lecture 15: Dimension reduction

Jamie Haddock

Table of contents

0.1 Rayleigh quotient 1
0.2 Definiteness 2

1 Dimension reduction **3**
1.1 SVD Optimality 3
1.2 Compression 4
1.3 Capturing major trends 6

0.1 Rayleigh quotient

For a matrix $\mathbf{A}$ and compatible vector $\mathbf{x}$, $\mathbf{x}^*\mathbf{A}\mathbf{x}$ is scalar-valued. This type of function is called a **quadratic form**.

> Definition: Rayleigh quotient
>
> Given symmetric (Hermitian) $\mathbf{A}$ and nonzero vector $\mathbf{x}$, the **Rayleigh quotient** is the function
>
> $$R_{\mathbf{A}}(\mathbf{x}) = \frac{\mathbf{x}^*\mathbf{A}\mathbf{x}}{\mathbf{x}^*\mathbf{x}}.$$

If $\mathbf{v}$ is an eigenvector such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, then one easily calculates that $R_{\mathbf{A}}(\mathbf{v}) = \lambda$. That is, the Rayleigh quotient maps an eigenvector into its associated eigenvalue.

If $\mathbf{A}^* = \mathbf{A}$, then the Rayleigh quotient has another interesting property: $\nabla R_{\mathbf{A}}(\mathbf{v}) = 0$ if $\mathbf{v}$ is an eigenvector. By a multidimensional Taylor series, then,

$$R_{\mathbf{A}}(\mathbf{v} + \epsilon\mathbf{z}) = R_{\mathbf{A}}(\mathbf{v}) + 0 + O(\epsilon^2) = \lambda + O(\epsilon^2),$$

as $\epsilon \to 0$.

A good estimate of an eigenvector becomes an even better estimate of an eigenvalue!

```
%%julia
n = 20;
 = 1:n
D = diagm( )
V,_ = qr(randn(n,n))   # get a random orthogonal V
A = V*D*V';
```

```
%%julia
R = x -> (x'*A*x)/(x'*x);  # define RQ
```

```
%%julia
R(V[:,7])   # evaluating the RQ at an evector gives corresponding evalue
```

```
7.0
```

```
%%julia
# RQ is within O(del^2) of the evalue, given a del perturbation to the evector
  = @. 1 ./10^(1:5)
eval_diff = zeros(size( ))
for (k,delta) in enumerate( )
    e = randn(n);  e = delta*e/norm(e);
    x = V[:,7] + e
    eval_diff[k] = R(x) - 7
end
labels = ["perturbation  "," ²","R(x) -  "]
pretty_table([  .^2 eval_diff], column_labels = labels)
```

```
┌ perturbation          ²      R(x) -
├
│           0.1      0.01     0.0319439
│          0.01    0.0001   0.000320629
│         0.001    1.0e-6    5.93454e-6
│        0.0001    1.0e-8    4.56097e-8
│        1.0e-5   1.0e-10     5.377e-10
└
```

0.2 Definiteness

In the real case, we called a symmetric matrix $\mathbf{A}$ *symmetric positive definite* (SPD) if $\mathbf{x}^\top \mathbf{A}\mathbf{x} > 0$ for all nonzero vectors $\mathbf{x}$.

> Theorem:
>
> If $\mathbf{A}^\top = \mathbf{A}$, then the following statements are equivalent.
> 1. $\mathbf{A}$ is SPD.
> 2. The eigenvalues of $\mathbf{A}$ are positive numbers.

Proof:

Suppose item 1 is true. If $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ is an eigenpair, then a Rayleigh quotient implies that $\mathbf{x}^\top\mathbf{A}\mathbf{x} = \lambda\mathbf{x}^\top\mathbf{x} = \lambda\|\mathbf{x}\|^2$ and thus, $\lambda > 0$. Hence item 2 is true.

Conversely, suppose item 2 is known. Then we can write the EVD as $\mathbf{A} = \mathbf{V}\mathbf{S}^2\mathbf{V}^*$, where the S_{ii} are positive square roots of the eigenvalues. Hence

$$\mathbf{x}^*\mathbf{A}\mathbf{x} = \mathbf{x}^*\mathbf{V}\mathbf{S}^2\mathbf{V}^*\mathbf{x} = \|\mathbf{S}\mathbf{V}^*\mathbf{x}\|^2 > 0,$$

as both $\mathbf{S}$ and $\mathbf{V}$ are invertible. Thus, item 1 is true.

For a SPD matrix, the EVD $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^*$ meets all the requirements of the SVD, provided the ordering of eigenvalues is chosen appropriately. They coincide!

A symmetric matrix with all negative eigenvalues is called **negative definite**, and one with eigenvalues of different signs is **indefinite**. Finally, if one or more eigenvalues is zero and the rest have one sign, it is positive or negative **semidefinite**.

1 Dimension reduction

The SVD provides an optimal rank-constrained approximation to a given matrix – this proves very useful in a variety of applications.

1.1 SVD Optimality

Let $\mathbf{A}$ be a real $m \times n$ matrix with SVD $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ and (momentarily) $m \geq n$. Another way of writing the thin form of the SVD is

$$\begin{aligned}
\mathbf{A} = \hat{\mathbf{U}}\hat{\mathbf{S}}\mathbf{V}^T &= \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \\
&= \begin{bmatrix} \sigma_1\mathbf{u}_1 & \cdots & \sigma_n\mathbf{u}_n \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix} \\
&= \sigma_1\mathbf{u}_1\mathbf{v}_1^T + \cdots + \sigma_r\mathbf{u}_r\mathbf{v}_r^T = \sum_{i=1}^{r} \sigma_i\mathbf{u}_i\mathbf{v}_i^T,
\end{aligned}$$

where r is the rank of $\mathbf{A}$. The final formula also holds for the case $m < n$.

Each outer product $\mathbf{u}_i\mathbf{v}_i^T$ is a rank-1 matrix of unit 2-norm. Thanks to the ordering of singular values, this expresses $\mathbf{A}$ as a sum of decreasingly important contributions.

This motivates the definition, for $1 \leq k \leq r$,

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i\mathbf{u}_i\mathbf{v}_i^T = \mathbf{U}_k\mathbf{S}_k\mathbf{V}_k^T,$$

where $\mathbf{U}_k$ and $\mathbf{V}_k$ are the first k columns of $\mathbf{U}$ and $\mathbf{V}$, respectively, and $\mathbf{S}_k$ is the upper-left $k \times k$ submatrix of $\mathbf{S}$.

The rank of a sum of matrices is always less than or equal to the sum of the ranks, so $\mathbf{A}_k$ is a rank-k approximation to $\mathbf{A}$. It turns out that $\mathbf{A}_k$ is the *best* rank-k approximation of $\mathbf{A}$, as measured in the matrix 2-norm.

Theorem:

Suppose $\mathbf{A}$ has rank r and let $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be an SVD. Let

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i\mathbf{u}_i\mathbf{v}_i^T = \mathbf{U}_k\mathbf{S}_k\mathbf{V}_k^T$$

for $1 \leq k < r$. Then

1. $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}, \quad k = 1, \ldots, r-1$, and
2. If the rank of $\mathbf{B}$ is k or less, then $\|\mathbf{A} - \mathbf{B}\|_2 \geq \sigma_{k+1}$.

Proof of part 1:

Note that the definition of $\mathbf{A}_k$ is identical to the SVD of $\mathbf{A}$ with $\sigma_{k+1}, \ldots, \sigma_r$ all set to zero. This implies that

$$\mathbf{A} - \mathbf{A}_k = \mathbf{U}(\mathbf{S} - \hat{\mathbf{S}})\mathbf{V}^T,$$

where $\hat{\mathbf{S}}$ has those same values of σ_i replaced by zero. But that makes the above an SVD of $\mathbf{A} - \mathbf{A}_k$, with singular values $0, \dots, 0, \sigma_{k+1}, \dots, \sigma_r$, the largest of which is σ_{k+1}. That proves the first claim.

1.2 Compression

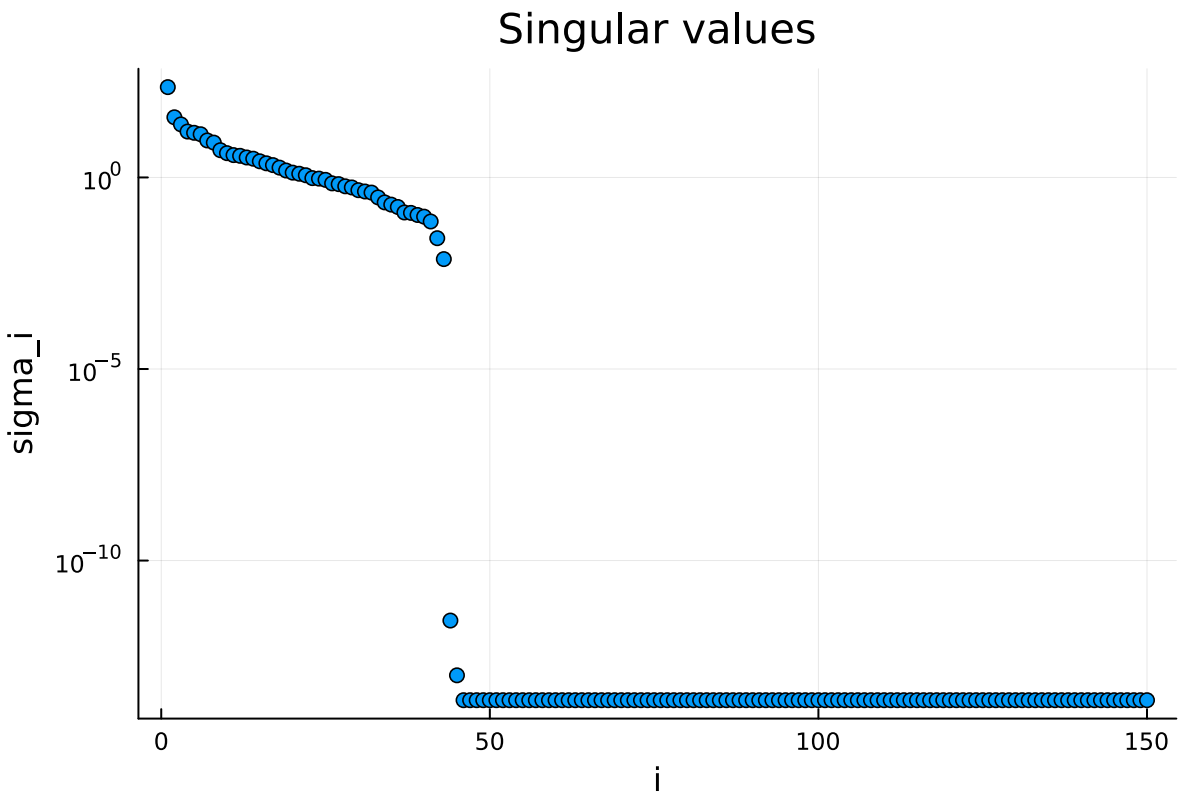
If the singular values of $\mathbf{A}$ decrease sufficiently rapidly, then $\mathbf{A}_k$ may capture the most significant behavior of the matrix for a reasonably small value of k.

```
%%julia
# make an image from text and reload it as matrix
gr()

Plots.plot(annotations=(0.5,0.5,text("Hello world",44,:center,:center)),
    grid=:none,frame=:none,size=(400,150))
Plots.savefig("hello.png")
img = load("hello.png")
A = @. Float64(Gray(img))
Plots.plot(Gray.(A),frame=:none)
```



```
%%julia
# singular values decrease until they reach numerical zero at k = 45
U, ,V = svd(A)
Plots.scatter( ,xaxis=("i"), yaxis=(:log10,"sigma_i"),
    title="Singular values",legend=false)
```



Rapid decrease suggests that low-rank approximations can be fairly accurate!

```
%%julia
plt = Plots.plot(layout=(2,2),frame=:none,aspect_ratio=1,titlefontsize=10)
for i in 1:4
    k = 3i
    Ak = U[:,1:k]*diagm( [1:k])*V[:,1:k]'
    Plots.plot!(Gray.(Ak),subplot=i,title="rank = $k")
end
plt
```




```
%%julia
m,n = size(A)  # compression ratio for rank-9 approximation
compression = m*n / (9*(m+n+1))
```

```
12.099213551119178
```

1.3 Capturing major trends

The use of dimension reduction offered by low-rank SVD approximation can uncover deep connections and trends that are otherwise obscured by weaker effects and noise.

One useful way to quantify the decay in the singular values is to compute

$$s_k = \sum_{i=1}^{k} \sigma_i^2, \quad \tau_k = \frac{s_k}{s_r}, \quad k = 1, \ldots, r.$$

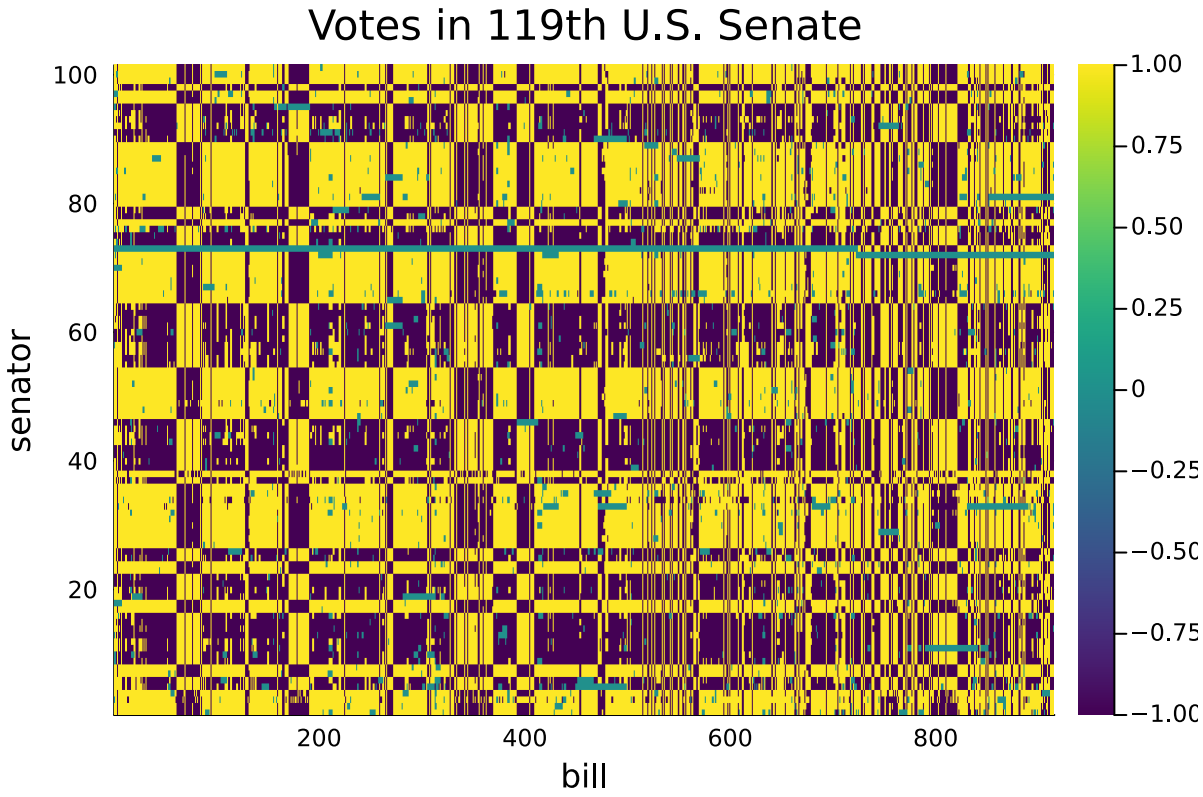
Clearly $0 \leq \tau_k \leq 1$ and τ_k is non-decreasing as a function of k. We can think of τ_k as the fraction of energy (or in statistical terms, variance) contained in the singular values up to and including the kth.

Example: This matrix data describes the votes on bills in the 119th session of the United States Senate. (The data set was obtained from [https://voteview.com].) Each row is one senator, and each column is a vote item.

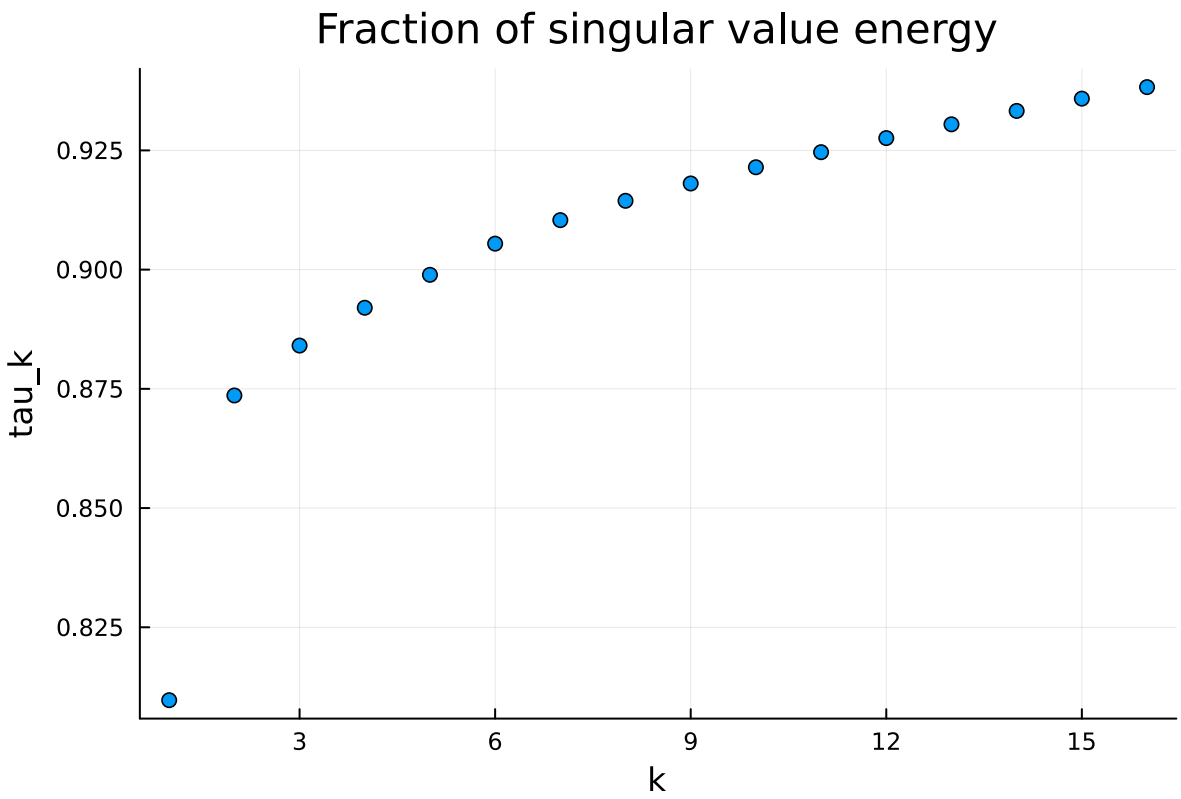
```
%%julia
# S119_members.csv and S119_votes.csv are from https://voteview.com (119th Congress, Senate)
using DelimitedFiles
mem,_ = readdlm("S119_members.csv", ',', header=true)
vts,_ = readdlm("S119_votes.csv", ',', header=true)
mem = mem[mem[:,2] .== "Senate", :]        # drop the President
```

```
icpsr = Int.(mem[:,3]);  party = Int.(mem[:,7]);  names = string.(mem[:,10])
row = Dict(id => i for (i,id) in enumerate(icpsr))
rolls, ids, cast = Int.(vts[:,3]), Int.(vts[:,4]), Int.(vts[:,5])
A = zeros(length(icpsr), maximum(rolls))
for (id,r,c) in zip(ids,rolls,cast)
    haskey(row,id) && (A[row[id],r] = c in 1:3 ? 1.0 : (c in 4:6 ? -1.0 : 0.0))  # yea = 1, nay = -1, el
end
keep = vec(sum(A .!= 0, dims=2)) .>= 0.1*size(A,2)   # drop senators who served very briefly
A = A[keep,:];  party = party[keep];  names = names[keep]
Dem = party .== 100;  Rep = party .== 200;  Ind = party .== 328;
```

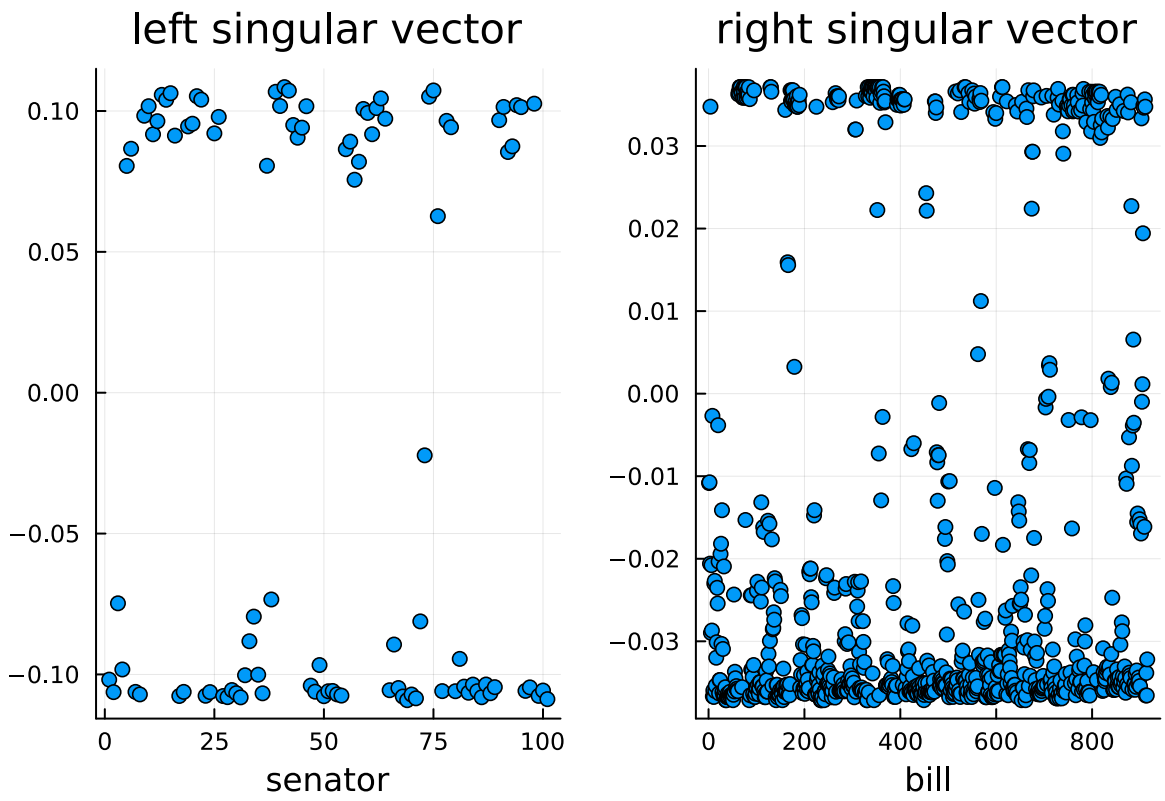
```
%%julia
Plots.heatmap(A,color=:viridis,
    title="Votes in 119th U.S. Senate",xlabel="bill",ylabel="senator")
    # yellow is yea, blue is nay
```



```
%%julia
U, ,V = svd(A)
  = cumsum( .^2) / sum( .^2)
Plots.scatter( [1:16], xaxis=("k"), yaxis=("tau_k"),
    title="Fraction of singular value energy",legend=false)
# first and second singular triples (2 vectors and 1 value)
# contain 81% and 6% of the energy in this dataset
```



```
%%julia
Plots.scatter( U[:,1],label="",layout=(1,2),
    xlabel="senator" ,title="left singular vector")
Plots.scatter!( V[:,1],label="",subplot=2,
    xlabel="bill",title="right singular vector")
# roughly speaking these vectors indicate how partisan a bill or senator is
```



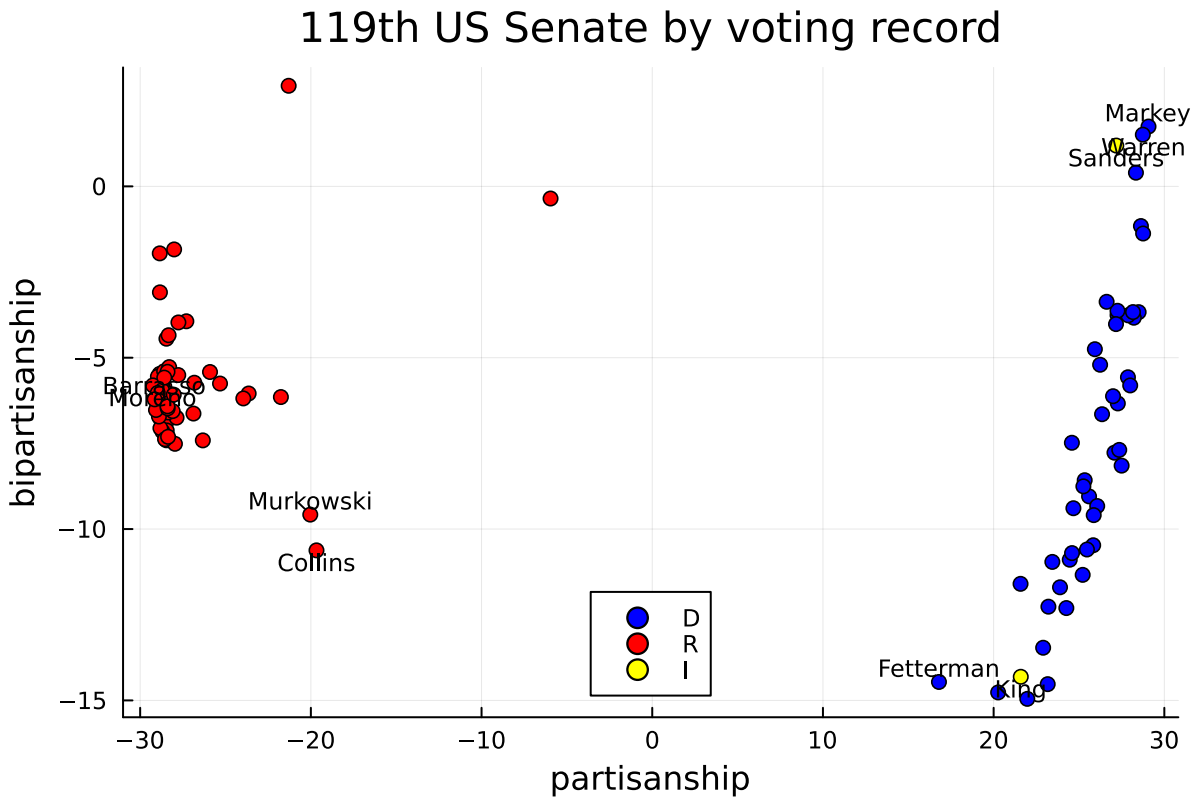
```
%%julia
x1 = A*V[:,1];   x2 = A*V[:,2];

# Live (Jupyter/VS Code): interactive hover plot.
# SET "interactive = true" to see the senator's name when hovering over a point.
interactive = false

if interactive
    plotly()   # hover over a point to see the senator's name
    plt = Plots.scatter(x1[Dem],x2[Dem],color=:blue,label="D", hover=names[Dem],
        xaxis=("partisanship"),yaxis=("bipartisanship"),title="119th US Senate by voting record",
        legend=:bottom)
    Plots.scatter!(x1[Rep],x2[Rep],color=:red,label="R", hover=names[Rep])
    Plots.scatter!(x1[Ind],x2[Ind],color=:yellow,label="I", hover=names[Ind])
else
    gr()       # static backend, so the figure appears in the html/pdf
    plt = Plots.scatter(x1[Dem],x2[Dem],color=:blue,label="D",
        xaxis=("partisanship"),yaxis=("bipartisanship"),title="119th US Senate by voting record",
        legend=:bottom)
    Plots.scatter!(x1[Rep],x2[Rep],color=:red,label="R")
    Plots.scatter!(x1[Ind],x2[Ind],color=:yellow,label="I")

    # labels: last name => where to put the text relative to the point
    labels = Dict("WARREN"=>:top, "MARKEY"=>:bottom,        # far Democrats
                  "FETTERMAN"=>:bottom,                     # moderate Democrat
                  "MORENO"=>:top, "BARRASSO"=>:bottom,      # far Republicans
                  "COLLINS"=>:top, "MURKOWSKI"=>:bottom,    # middle Republicans
```

```
                  "SANDERS"=>:top, "KING"=>:top)             # independents
    for i in eachindex(names)
        ln = uppercase(first(split(names[i], ",")))
        haskey(labels, ln) && Plots.annotate!(x1[i], x2[i],
            Plots.text(titlecase(ln), 8, :center, labels[ln]))
    end
end
plt
```



Projecting the senators' vectors into the first two $\mathbf{V}$-coordinates gives a particularly nice way to reduce them to two dimensions. Political scientists label these dimensions *partisanship* and *bipartisanship*. Here we color them by actual party affiliation (also given in the data file): red for Republican, blue for Democrat, and yellow for independent.

Not all data sets can be reduced effectively to a small number of dimensions, but as this demo illustrates, in some cases dimension reduction reveals information that corresponds to real-world understanding.